

Executing SQL Stored Procedures from inside a Web Application, Part 3

By Don Schlichting

Introduction

This article focuses on using a stored procedure to insert a new record from data collected on a web page, then passing an identity value created from that row back to the web page. In the previous articles in this series, [Part One](#) described the benefits of using stored procedures along with an example of populating a web data grid from a stored procedure. [Part Two](#) expanded by introducing INPUT parameters, which were used in the WHERE clause of a stored procedure. The value of the INPUT was gathered from a text box on a web page. This article continues where Part One and Two left off, working with OUTPUT parameters.

Parameters

There are three types of parameters that can be passed into or from a stored procedure: INPUT, OUTPUT, and RETURN. INPUT parameters are used to pass data into a procedure from a calling application, OUTPUT parameters pass data from a procedure back to the calling application, and RETURN passes an error or completion code back from the procedure.

OUTPUT

For this article, we are interested in having a procedure create a new row, which includes an auto identity field, and getting that new identity back to our calling application (a web page in this example). This is a perfect fit for an OUTPUT parameter. An OUTPUT parameter is used to send a value back to the application that called it. Create a test database and table with the following script.

```
CREATE DATABASE test;
GO
USE test;
GO
CREATE TABLE Table1
(
  ID int IDENTITY(1,1),
  DateCreated datetime
);
```

Table1 contains an auto identity field, directing SQL to auto number the ID field for every row created. This is the value we will want to return to a web page. The following script will create a stored procedure that will populate the DateCreated field with the date and time the procedure was executed.

```
CREATE PROCEDURE OutputTest
(
  @ID int OUTPUT
)
AS
```

```

INSERT INTO Table1
(DateCreated)
VALUES
( GETDATE() )
SET @ID = SCOPE_IDENTITY()
;

```

Notice that our parameter @ID is placed above the AS key word and without using a DECLARE statement (Identical to the INPUT parameters from the previous articles). But now, it must be followed by the keyword OUTPUT. The last statement sets the parameter to the value just inserted by the auto identity column. There are two SQL functions for returning a newly inserted identity; SCOPE_IDENTITY and @@IDENTITY. One of the main differences between the two is that its SCOPE_IDENTITY is guaranteed to return the value just created by the stored procedure it is run inside. In other words, its scope is local to the procedure calling it. Using @@IDENTITY, it is possible to get an identity from some other SQL table if triggers are used in the database. See [Getting the Wrong Identity in Microsoft SQL Server identity Columns?](#) for a complete explanation.

Test the new procedure by executing it. One row should be returned. The zero at the end of the EXEC statement is a placeholder for the OUTPUT parameter.

```

EXEC OutputTest 0;
SELECT * FROM Table1;

```

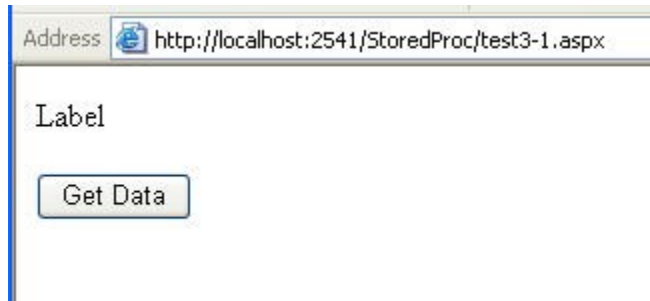
	ID	DateCreated
1	1	2006-04-26 21:07:50.017

Next, we will create a web page with a single button. Its click event will execute the stored procedure created above. After the procedure creates a new row, it will return the ID value of that row to the calling web page where it will be displayed as a label.

```

<%@ Page Language="C#" %>
<html xmlns="http://www.w3.org/1999/xhtml" >
<head id="Head1" runat="server">
    <title>Test 1</title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <p><asp:Label ID="Label1" runat="server" Text="Label"></asp:Label> </p>
            <p><asp:Button ID="btnExecute" runat="server" Text="Get Data"
OnClick="Execute_Clicked" /></p>
        </div>
    </form>
</body>
</html>

```



The following click event code will tie the button to the stored procedure using C#.

```
<script runat="server">
protected void Execute_Clicked(object sender, EventArgs e)
{
    System.Data.SqlClient.SqlConnection objConn = new
    System.Data.SqlClient.SqlConnection();
    objConn.ConnectionString = "server=.; database=test; uid=sa; pwd=test;";
    objConn.Open();
    System.Data.SqlClient.SqlCommand objCmd = new
    System.Data.SqlClient.SqlCommand("OutputTest", objConn);
    objCmd.CommandType = System.Data.CommandType.StoredProcedure;
    System.Data.SqlClient.SqlParameter pID = objCmd.Parameters.Add("@ID",
    System.Data.SqlDbType.Int);
    pID.Direction = System.Data.ParameterDirection.Output;
    objCmd.ExecuteScalar();
    objConn.Close();
    Label1.Text = objCmd.Parameters["@ID"].Value.ToString();
}
</script>
```

Here is the same event but in VB.

```
<script runat="server">
Protected Sub Execute_Clicked(ByVal sender As Object, ByVal e As EventArgs)
Dim objConn As System.Data.SqlClient.SqlConnection = New
System.Data.SqlClient.SqlConnection()
objConn.ConnectionString = "server=.; database=test; uid=sa; pwd=test;"
objConn.Open()

Dim objCmd As System.Data.SqlClient.SqlCommand = New
System.Data.SqlClient.SqlCommand("OutputTest",objConn)
objCmd.CommandType = System.Data.CommandType.StoredProcedure

Dim pID As System.Data.SqlClient.SqlParameter =
objCmd.Parameters.Add("@ID",System.Data.SqlDbType.Int)
pID.Direction = System.Data.ParameterDirection.Output

objCmd.ExecuteScalar()
objConn.Close()

Label1.Text = objCmd.Parameters("@ID").Value.ToString()
```

```
End Sub  
</script>
```

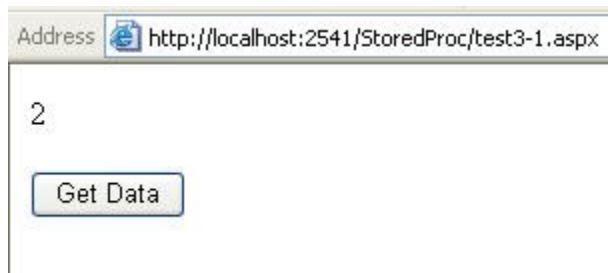
Working with the OUTPUT parameter is very similar to the INPUT parameter example from the previous article. The first three lines open a connection to the database. The next two lines specify that a stored procedure will be executed and its name. These next two lines specify the parameter.

```
System.Data.SqlClient.SqlParameter pID = objCmd.Parameters.Add("@ID",  
System.Data.SqlDbType.Int);  
pID.Direction = System.Data.ParameterDirection.Output;
```

Here we have defined a parameter, and specified its of the type OUTPUT. The parameter name @ID must match the name used in the stored procedure. If it does not, this error will be thrown at runtime "Procedure or Function 'OutputTest' expects parameter '@ID', which was not supplied. ". Notice we do not need a placeholder for the value such as pID.VALUE = 0 like we did when the procedure was executed from Query Analyzer.

Next, the command is executed then the connection closed. The last line moves the value of OUTPUT parameter to the label. This will be the value created by stored procedure line "SET @ID = SCOPE_IDENTITY()".

Clicking the button on the web page demonstrates the stored procedure created a new row, ID 2, and its value was successfully returned to the web page.



Conclusion

This example demonstrated how to execute a stored procedure and get a value returned to the calling program. It is a great fit for an application that collects information from a user, and then creates a new record, but after additional data needs to be appended to that new record from the next web page or Windows Form. The ID of the new record can be sent back to the calling application then tied to a global variable, cookie, or passed in a string to the next page. In addition, as stored procedures are used over writing TSQL in the application and parsing it at runtime, all the benefits of code separation will begin to materialize. Some of the same benefits we get by using a web Code Behind method are also delivered by separating our database code out of the web page and placing it in SQL as a stored procedure. In the next article, we will continue by examining how SQL can send both record sets and parameters at the same time, as well as introducing RETURN parameters.